

NoC-based CNN Accelerator with AXI4-like DMA and SRAM-assisted Systolic FC Dataflow

Group 6 : Chia-Hung Chou
Machine Learning Intelligent Chip Design 114-2

Abstract—This final project implements a SystemC-based CNN accelerator with a baseline and an optimized architecture. The baseline replaces direct file/ROM-style tensor access with an AXI4-like DMA and DRAM model while using a controller-scheduled PE execution style at a mostly behavioral abstraction level. The optimized design introduces explicit global SRAM, PE-local SRAM, DMA-to-SRAM data movement, bounded controller registers, and a PE-level systolic fully-connected dataflow mapped onto the physical 4×4 NoC mesh. The optimized FC classifier uses the 16 PEs as two 2×4 MVM sub-arrays, overlaps FC weight prefetch with computation through ping-pong SRAM buffers, and commits final scores back to DRAM before output reporting.

I. INTRODUCTION

The baseline design uses a simple controller-scheduled dataflow and many behavioral abstractions for tensor staging and PE computation. For the final project, the memory system is remodeled as a DRAM accessed through an AXI4-like DMA. The design goal is twofold. First, the baseline provides a functionally correct reference design that executes AlexNet through DMA-backed DRAM transfers. Second, the optimized version exposes a more hardware-meaningful memory hierarchy and improves the fully-connected (FC) classifier by using SRAM tiling, DMA prefetch, and PE-level systolic execution.

The target workload is AlexNet inference. As summarized in Fig. 1, the network first pads the $224 \times 224 \times 3$ input image to $227 \times 227 \times 3$, then executes five convolutional stages with ReLU activations and three max-pooling stages. These layers reduce the spatial feature map to $6 \times 6 \times 256$. The classifier then flattens this feature map into a 9216-element vector and applies three FC layers with output sizes 4096, 4096, and 1000. Finally, the softmax layer converts the 1000 logits into classification probabilities. This layer structure determines the accelerator schedule: convolution and pooling layers are dispatched as PE jobs, while the large FC matrix-vector multiplications are the main target of the optimized SRAM-assisted systolic dataflow.

II. OVERALL OPTIMIZED ARCHITECTURE

Fig. 2 shows the optimized accelerator architecture. The system contains an external DRAM, an AXI4-like DMA, a lightweight controller, a global SRAM scratchpad, a 4×4 NoC mesh, and 16 PE workers. Each PE contains a local SRAM model, a 64-lane MAC array model, and a nonlinear function block.

The DRAM side is accessed by an AXI4-like interface. The implemented signals include AWADDR, AWLEN, AWVALID, A

WREADY, WDATA, WLAST, WVALID, WREADY, BRESP, BVALID, BREADY, ARADDR, ARLEN, ARVALID, ARREADY, RDATA, RLAST, RVALID, and RREADY. The AXI data bus is modeled as a 32-bit `sc_uint<32>` word carrying IEEE-754 single-precision floating-point bit patterns. Optional full AXI4 sideband signals such as ID, cache, protection, QoS, user, and out-of-order response support are omitted because the current accelerator issues one simplified DMA transaction at a time.

III. BASELINE ARCHITECTURE

The baseline keeps a simple controller-staged computation flow. The controller reads tensors from DRAM using the DMA, stages the received stream into controller-side behavioral vectors, partitions layer work into PE jobs, sends NoC packets to worker PEs, and collects result packets. PE0 is not used as a worker in the baseline because router node 0's local port is occupied by the controller. Therefore, the baseline uses 15 worker PEs.

A. Baseline Dataflow

For DRAM reads, the controller writes a DMA command containing the source byte address and transfer length. The DMA becomes the AXI master, sends AR channel requests to DRAM, receives R channel data beats, converts each 32-bit bus word back to a floating-point value, and streams values to the controller using `read_valid/read_ready`. For DRAM writes, the controller streams values to DMA through `write_valid/write_ready`; the DMA then performs AW, W, and B channel transactions to DRAM.

After tensors are available in the controller-side behavioral buffer, the controller dispatches PE jobs through the NoC. A packet contains one header flit followed by packed payload flits. The packet payload describes the operation type, layer id, tensor tile, job id, and other operation-specific fields. The PE computes the assigned convolution, pooling, or FC job and returns a packet to the controller.

The baseline is intentionally simple. Its controller-side tensor vectors behave like large staging buffers. This is acceptable for the baseline reference because the baseline is meant to be a functional reference at a higher behavioral abstraction level, but it is not ideal as a hardware optimization. The optimized design therefore removes this hidden large buffer from the top-level conv, pool, and FC schedule by making activations, weights, biases, partial sums, and outputs SRAM-resident.

Layer	Type	Description	Output Dimension
0	Input Layer	Zero Padding 224 to 227	227x227x3
1	Convolutional Layer 1 + ReLU	64 kernels of 11x11, stride 4	55x55x64
1b	Max Pooling 1	3x3, stride 2	27x27x64
2	Convolutional Layer 2 + ReLU	192 kernels of 5x5, stride 1, padding 2	27x27x192
2b	Max Pooling 2	3x3, stride 2	13x13x192
3	Convolutional Layer 3 + ReLU	384 kernels of 3x3, stride 1, padding 1	13x13x384
4	Convolutional Layer 4 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5	Convolutional Layer 5 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5b	Max Pooling 5	3x3, stride 2	6x6x256
6	Fully Connected Layer 6 + ReLU	4096 neurons	4096
7	Fully Connected Layer 7 + ReLU	4096 neurons	4096
8	Fully Connected Layer 8	1000 neurons	1000
9	Softmax Layer	Converts logits to probabilities	1000

Fig. 1: AlexNet inference schedule used by the final project. The convolution and pooling stages extract spatial features, while the final three FC layers dominate the classifier matrix-vector multiplication workload.

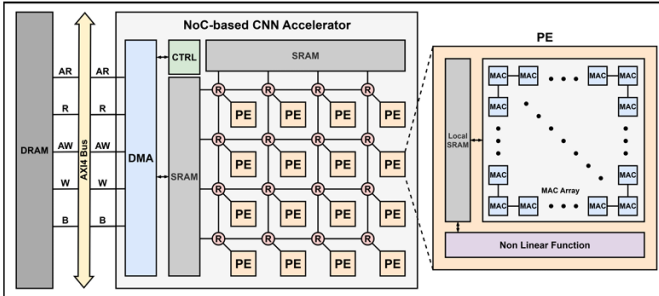


Fig. 2: Optimized NoC-based CNN accelerator architecture with AXI4-like DMA, global SRAM, PE-local SRAM, and PE-level systolic FC dataflow.

IV. OPTIMIZED ARCHITECTURE

A. Controller as a Control Processor

In the optimized design, the controller is modeled as a lightweight accelerator control processor rather than as a large tensor memory. Its hardware-meaningful state consists of DMA descriptor/status registers, loop counters, NoC packet serialization state, and an 8-word FC tile register file. The 8-word tile register corresponds to the maximum number of output neurons computed by one FC tile. It is used for bias, partial-sum, output-tile, and result bookkeeping values. Large

FC tensors are stored in global SRAM rather than controller vectors.

B. DMA and Global SRAM Data Movement

The optimized DMA service supports four descriptor modes:

- DRAM to controller stream, retained for compatibility and small diagnostic paths.
- Controller stream to DRAM, retained for the output writeback test.
- DRAM to global SRAM, used by optimized FC weight and bias staging.
- Global SRAM to DRAM, used by final FC8 output writeback.

During FC execution, the controller programs DMA descriptors but does not move bulk tensor words itself. For a weight tile, the controller sets the DRAM source address, the global SRAM destination address, and the number of words. The DMA then reads DRAM through AXI and writes each received word into global SRAM. For final output writeback, the DMA reads the global SRAM output region and writes the scores back to DRAM. The final scores are then read back before softmax and Top-100 printing, ensuring that reported results come from committed DRAM output.

TABLE I: DRAM address map used by both designs.

Region	Base Address	Purpose
Input image	0x00000000	Test image tensor
Conv1 W/B	0x00100000/0x00200000	Conv1 parameters
Conv2 W/B	0x00300000/0x00700000	Conv2 parameters
Conv3 W/B	0x00800000/0x01000000	Conv3 parameters
Conv4 W/B	0x01100000/0x01b00000	Conv4 parameters
Conv5 W/B	0x01c00000/0x02300000	Conv5 parameters
FC6 W/B	0x02400000/0x1a000000	FC6 parameters
FC7 W/B	0x1a100000/0x24b00000	FC7 parameters
FC8 W/B	0x24c00000/0x27600000	FC8 parameters
Output	0x27700000	Final 1000 scores

TABLE II: Optimized global SRAM address map.

Region	Word Base	Purpose
FC input	0	FC activation input buffer
FC weight ping	65536	Weight tile buffer 0
FC weight pong	131072	Weight tile buffer 1
FC bias tile	196608	Bias tile staging
FC psum	200704	Partial-sum tile storage
FC output	204800	FC output / final scores
Conv buffer A	0	Conv/pool ping buffer
Conv buffer B	262144	Conv/pool pong buffer
Conv weight	524288	Conv weight staging
Conv bias	1500000	Conv bias staging

C. Global SRAM and PE-local SRAM

The optimized design uses an 8 MB global SRAM scratchpad. FC activations are ping-ponged between two SRAM regions: one region acts as the input activation buffer and the other as the output activation buffer. FC6 writes to the output region, FC7 reads that region and writes back to the input region, and FC8 reads the input region and writes final scores to the output region.

Each PE also contains a 64 KB local SRAM model. The PE-local SRAM stores input segments, weight segments, and small tile data needed by the PE. This gives the PE compute path a clear hardware memory structure instead of relying on unbounded software vectors.

D. Memory Layout Design

The external DRAM is byte-addressed, and all tensors are stored as contiguous 32-bit floating-point words. Both the baseline and optimized designs use the same DRAM layout so that the comparison uses identical input, weight, bias, and output placement. Table I summarizes the major DRAM regions.

The optimized design additionally defines an on-chip global SRAM address map, shown in Table II. Convolution and pooling layers ping-pong feature maps between two SRAM buffers, while FC layers use separate regions for input activations, ping-pong weight tiles, bias tiles, partial sums, and output scores. The final FC8 output region is transferred back to DRAM_OUTPUT_BASE before softmax and Top-100 reporting.

E. NoC Operation and Controller Dispatch

The NoC is a custom valid/ack packet network rather than AXI or APB. Each flit contains a flit type, a destination/source header field, and packed 32-bit payload lanes. The controller serializes each packet into one HEAD flit and

multiple BODY/TAIL flits. Routers use two virtual channels: VC0 is a deadlock-free deterministic XY escape path, while VC1 uses adaptive minimal routing to reduce blocking when possible. The receiver reconstructs packets and acknowledges each flit using the valid/ack handshake.

For convolution and pooling, the controller keeps a simple channel-partitioned PE dispatch scheme, but the bulk tensors are stored in Global SRAM. The controller uses SRAM addresses to stream input, weight, and bias words directly into NoC flit lanes, sends those flits to PE-local SRAM, dispatches compute commands, and writes returned result slices back to the Global SRAM output region. For optimized FC, the dispatch is more structured: the controller stages activations and weights through SRAM, then sends input segments and weight segments to the PE rows/columns participating in the systolic MVM tile.

F. Systolic FC Dataflow

The optimized FC classifier maps matrix-vector multiplication onto the physical 4×4 PE mesh. The mesh is treated as two independent 2×4 sub-arrays. Each sub-array computes up to four output neurons, so one FC tile computes up to eight output neurons.

For an FC layer with input vector b and weight matrix A , each output is:

$$c_i = \sum_j a_{ij}b_j + bias_i. \quad (1)$$

The input dimension is tiled. For each input tile, the input vector segment $b(j)$ is read from global SRAM and broadcast horizontally across the active PE row. Weight segments $a(i, j)$ are prefetched from DRAM to global SRAM using ping-pong buffers, then loaded into PE-local SRAM. Partial sums move vertically from the bottom PE row to the top boundary row inside each 2×4 sub-array. The top-boundary PE returns the updated partial sum to the controller tile register. After the last input tile, ReLU is applied when required.

G. DMA Prefetch and Compute Overlap

The optimized FC path uses ping-pong SRAM buffers for FC weight tiles. While the PE array computes input tile k using one weight buffer, the controller starts a DMA-to-SRAM prefetch for input tile $k+1$ into the other buffer. This prevents the active compute tile from being overwritten and allows memory movement to overlap with systolic computation. The controller only synchronizes when the next tile is actually needed.

V. CORE CONTRIBUTIONS OF THE OPTIMIZED DESIGN

- 1) **Recovered the full 4×4 PE array.** In the baseline, router node 0 is occupied by the controller endpoint, so only worker PEs 1–15 are used for CNN computation. This baseline organization is simple, but one physical mesh node is not available as a compute PE. The optimized design moves the controller to a dedicated router HOST port and restores PE0 as a normal worker.

Therefore the optimized design uses all 16 PEs. With 64 modeled MAC lanes in each PE, the available MAC resources increase from $15 \times 64 = 960$ MAC lanes in the baseline to $16 \times 64 = 1024$ MAC lanes in the optimized design. The baseline trace reports 714,188,480 PE MAC operations across 120 compute jobs. Recovering the last PE increases the available compute fabric without changing the CNN workload and makes the optimized mesh match the intended 4×4 accelerator topology.

- 2) **Introduced an explicit SRAM hierarchy instead of implicit controller storage.** The baseline follows a controller-staged behavioral design: tensors are fetched from DRAM through DMA, assembled in controller-side software vectors, and then sent to PEs through the NoC. This is functionally correct, but large temporary vectors are less clear as hardware structures. The optimized design adds an 8 MB global scratchpad SRAM and 64 KB PE-local SRAM per PE, for a total on-chip SRAM capacity of 9 MB. The global SRAM is reported as two banks, while each PE-local SRAM models separate input and weight banking. This gives activations, weights, bias values, partial sums, and output tiles explicit hardware storage locations. This also makes memory movement measurable: the baseline reports 0 GB SRAM traffic because no explicit SRAM model exists, while the optimized run reports about 0.31 GB of SRAM reads and 0.25 GB of SRAM writes through the explicit scratchpad model. The baseline controller-side vectors are kept as a behavioral reference model with effectively unconstrained temporary storage, while the optimized design replaces this style with hardware-meaningful SRAM storage.
- 3) **Converted FC execution to an SRAM-resident dataflow.** In the baseline FC path, the controller loads the full FC input and PE weight slices through DMA and dispatches jobs after the data is available. The optimized FC path stores the input activation vector, FC weight tiles, bias tiles, partial sums, and output scores in the global SRAM address map. The controller keeps only a bounded 8-word tile register for the current output group, matching the eight outputs computed by the two 2×4 sub-arrays. This reduces the amount of persistent controller-side buffering and makes the FC storage behavior closer to a hardware scratchpad-based accelerator.
- 4) **Added DMA-to-SRAM and SRAM-to-DMA movement modes.** The baseline DMA moves data between DRAM and the controller streaming interface, so the controller is on the data path for every tensor transfer. The optimized DMA service adds descriptor modes that move DRAM data directly into global SRAM and move final SRAM output data back to DRAM. In the optimized trace, the final classifier writes only 1000 output-score words back to DRAM before printing, satisfying the project rule that output results must be written back to DRAM first. The recorded optimized

traffic is about 0.245 GB of DRAM reads and only 4000 bytes of DRAM writes for the final scores, matching the baseline DRAM traffic while giving the optimized design an explicit SRAM-resident execution path.

- 5) **Mapped FC matrix-vector multiplication onto PE-level systolic sub-arrays.** The baseline assigns FC work as independent PE jobs: each worker receives an input buffer, a weight slice, and a bias slice, then returns a contiguous output-neuron range. This provides parallelism, but it does not expose a structured systolic dataflow. The optimized design uses the physical 4×4 PE mesh as two 2×4 MVM sub-arrays. Input-vector words are broadcast horizontally, local weight segments are consumed from PE-local SRAM, and partial sums move vertically through each sub-array. The optimized log reports 58,621,952 systolic MAC operations, 2,173 input tiles, and an estimated FC systolic utilization of 86.82%.
- 6) **Overlapped FC weight prefetch with computation using ping-pong SRAM buffers.** In the baseline, a layer stage waits for DMA movement to complete before dispatching the corresponding PE computation, so memory movement, communication, and compute are largely serialized. The optimized FC path uses two global SRAM weight buffers. While the PE array computes with the current buffer, DMA prefetches the next weight tile into the other buffer. This prevents prefetch traffic from overwriting active compute data and creates explicit overlap between DRAM access and systolic computation. The optimized metrics separately report modeled SRAM wait cycles and modeled systolic wait cycles so this overlap can be discussed in the evaluation. In the reported optimized run, the SRAM model accounts for 139,478,774 modeled SRAM wait cycles and the FC systolic model accounts for 65,940 modeled systolic wait cycles.
- 7) **Made the PE model more hardware-structured.** The baseline PE is primarily a behavioral compute worker. The optimized PE still uses behavioral arithmetic for tractable SystemC simulation, but its internal organization is described as a 64-lane MAC array connected to PE-local SRAM and a nonlinear function block. This reflects the intended hardware partition: local SRAM supplies operands, the MAC array performs parallel multiply-accumulate operations, and the nonlinear block applies activation functions such as ReLU. The 64-lane setting is also used by the execution-cycle and utilization counters, so the performance model is tied to an explicit hardware resource count.
- 8) **Added end-of-run evaluation counters for reproducible comparison.** The baseline flow mainly demonstrates functional correctness with a simple behavioral control schedule. The baseline and optimized versions now both print metrics that can be inserted into the comparison table, including DRAM traffic, SRAM traffic when applicable, NoC payload words, packets, flits,

injection cycles, transfer latency, execution cycles estimated from the SystemC timestamp, and PE utilization estimates. The optimized run finishes at 1,520,800,720 ns, or about 152,080,072 simulated cycles with a 10 ns clock. For the optimized run shown in the trace, the NoC carried 76,789,835 payload words in 4,905,197 injected flits. With 16 words per flit, this corresponds to about 97.84% payload packing efficiency. The baseline payload packing from the measured run is about 99.98%. These counters make the baseline and optimized designs comparable even when the optimized design models more hardware structures.

VI. EVALUATION

The following commands produce the logs used to fill Table III:

```
cd Final_project/FP
make clean
make trace-cat

cd ../FP_Optimized
make clean
make trace-cat
```

At the end of the baseline log, use the section `BaselineDesignMetrics`. At the end of the optimized log, use the section `OptimizedDesignMetrics` and `GlobalScratchpadSRAMMetrics`. The reported byte counts can be converted to GB by dividing by 10^9 .

A. Recorded Counters

The simulator prints the following counters at the end of each run.

- Baseline DRAM counters: `baseline_dram_read_words`, `baseline_dram_write_words`.
- Baseline NoC counters: `baseline_noc_payload_words`, `baseline_noc_packets`, `baseline_noc_tx_flits`, `baseline_noc_tx_cycles`.
- Baseline PE compute counters: `PE::total_mac_ops()`, `PE::total_compute_cycles()`, `PE::total_compute_jobs()`.
- Baseline FC-only PE compute counters: `PE::total_fc_mac_ops()`, `PE::total_fc_compute_cycles()`, `PE::total_fc_compute_jobs()`.
- Optimized DRAM counters: `optimized_dram_read_words`, `optimized_dram_write_words`.
- Optimized NoC counters: `optimized_noc_payload_words`, `optimized_noc_packets`, `optimized_noc_tx_flits`, `optimized_noc_tx_cycles`.
- Optimized FC cycle counters: `optimized_sram_wait_cycles`, `optimized_systolic_wait_cycles`, `optimized_systolic_mac_ops`, `optimized_systolic_tiles`.
- Global SRAM counters: `global_sram.read_words`, `global_sram.write_words`, `global_sram.read_ops`, `global_sram.write_ops`, `global_sram.modeled_cycles`.

`read_ops`, `global_sram.write_ops`, `global_sram.modeled_cycles`.

B. Memory Traffic

All tensor values are single-precision floating-point values, so one word is 4 bytes. The DRAM traffic is computed as:

$$DRAM_{read,GB} = \frac{DRAM_{read,words} \times 4}{10^9}, \quad (2)$$

$$DRAM_{write,GB} = \frac{DRAM_{write,words} \times 4}{10^9}. \quad (3)$$

For the baseline, these word counters are `baseline_dram_read_words` and `baseline_dram_write_words`. For the optimized design, they are `optimized_dram_read_words` and `optimized_dram_write_words`. The SRAM traffic is only meaningful for the optimized design because the baseline does not model explicit SRAM storage. It is computed from the global SRAM counters:

$$SRAM_{read,GB} = \frac{SRAM_{read,words} \times 4}{10^9}, \quad (4)$$

$$SRAM_{write,GB} = \frac{SRAM_{write,words} \times 4}{10^9}. \quad (5)$$

The printed log fields are `SRAMreadbytes`, `SRAMwritebytes`, `SRAMreadGB`, and `SRAMwriteGB`. The total memory access volume is:

$$Mem_{total,GB} = DRAM_{read,GB} + DRAM_{write,GB} + SRAM_{read,GB} + SRAM_{write,GB}. \quad (6)$$

For the baseline, `SRAMread,GB` and `SRAMwrite,GB` are zero, so the total memory access volume is simply the DRAM read/write volume.

C. PE Utilization

For a fair comparison with the optimized FC systolic utilization, the main baseline utilization reported in Table III uses only FC-layer PE jobs. The baseline PE model records useful FC MAC operations every time an FC job calls the PE MAC wait model. It also records the corresponding active FC compute cycles:

$$C_{FC,base} = \sum_{FC \text{ jobs}} \left\lceil \frac{MAC_{job}}{64} \right\rceil. \quad (7)$$

The estimated baseline FC PE compute utilization is:

$$Util_{FC,base}(\%) = \frac{MAC_{FC,base}}{C_{FC,base} \times 64} \times 100. \quad (8)$$

This is an FC active-compute utilization estimate. It does not include DMA, NoC, Controller scheduling, or PE idle time in the denominator. The baseline log also prints a system-level PE utilization using the total SystemC timestamp; that number is useful for understanding idle time, but it is not the value used to compare against the optimized FC systolic utilization.

The optimized FC path records the number of useful MAC operations in `optimized_systolic_mac_ops`. It also records the modeled systolic execution cycles in `optimize`

`d_systolic_wait_cycles`. Since the optimized design uses 16 PEs and each PE models 64 MAC lanes, the peak number of available MAC operations over the modeled systolic cycles is:

$$MAC_{peak} = C_{systolic} \times 16 \times 64. \quad (9)$$

The estimated PE utilization is:

$$Utilization(\%) = \frac{MAC_{useful}}{MAC_{peak}} \times 100. \quad (10)$$

This utilization is an FC systolic utilization estimate, not a full-program PE activity ratio. Therefore, the baseline value and optimized value should be interpreted as compute-model utilization estimates rather than full-system PE idle/active ratios.

D. Execution Cycles

The SystemC simulation timestamp is printed as `SystemCtimestamp`. This is the global simulated time when inference finishes. For optimized FC, two additional cycle-aware counters are printed: `ModeledSRAMwaitcycles` and `Modeledsystolicwaitcycles`. The SRAM counter is accumulated from global SRAM read/write latency, while the systolic counter is accumulated from the FC tile compute model:

$$C_{tile} = \left\lceil \frac{TileWords}{MACsPerPE \times SubarrayRows} \right\rceil + SubarrayRows + ArrayCols - 2. \quad (11)$$

The optimized report therefore separates the global SystemC timestamp from the modeled SRAM and systolic FC cycle components.

E. NoC Utilization and Transfer Latency

The controller records NoC traffic at the packet injection interface. For every packet sent by the controller, the payload size is accumulated in `noc_payload_words`. For every flit injected into router 0, the simulator increments `noc_tx_flits`. The number of cycles spent waiting for valid/ack handshakes during flit injection is accumulated in `noc_tx_cycles`.

The NoC flit format packs up to 16 32-bit payload words in one BODY/TAIL flit. Therefore, the payload packing ratio reported in Table III is computed as:

$$NoC_{packing}(\%) = \frac{NoC_{payload,words}}{NoC_{tx,flits} \times 16} \times 100. \quad (12)$$

The average number of payload words per injected flit can also be estimated as:

$$NoC_{payload/flit} = \frac{NoC_{payload,words}}{NoC_{tx,flits}}. \quad (13)$$

If a cycle-normalized utilization is desired, the report can use:

$$NoC_{payload/cycle} = \frac{NoC_{payload,words}}{NoC_{tx,cycles}}. \quad (14)$$

The reported NoC transfer latency is the average controller-side injection latency per flit:

$$Latency_{NoC} = \frac{NoC_{tx,cycles}}{NoC_{tx,flits}}. \quad (15)$$

This is not a full router-by-router internal latency measurement. It is a stable end-to-end metric at the controller injection port and is used to compare the baseline and optimized traffic behavior under the same NoC handshake model.

VII. CONCLUSION

The baseline verifies the functional correctness of executing AlexNet through AXI4-like DMA and DRAM, but it remains a high-level behavioral reference. Its controller-side and PE-local temporary buffers hide many storage-capacity and data-staging costs, so several metrics, such as SRAM traffic and active FC PE utilization, appear optimistic.

The optimized design does not improve every measured metric. Its SystemC runtime, modeled execution cycles, and total modeled memory-access volume are higher because the design explicitly exposes costs that the baseline abstracts away. The main contribution is therefore hardware realism rather than raw simulation speedup: the optimized architecture introduces a concrete global SRAM, PE-local SRAM, DMA-to-SRAM and SRAM-to-DMA movement, ping-pong FC weight prefetch, bounded controller registers, and a PE-level systolic MVM dataflow. These structures make the memory hierarchy, data movement, and resource usage visible and measurable.

Overall, the optimized design should be interpreted as a more hardware-aware accelerator model. It trades the artificially favorable metrics of a high-level behavioral baseline for explicit resource accounting and a clearer mapping from SystemC modules to accelerator hardware structures.

TABLE III: Comparison between baseline and optimized designs. Runtime-dependent values are printed at the end of `make trace-cat` or `make trace-dog`.

Metric / Design Aspect	Baseline Design	Optimized Design	Notes / Log Field
Number of PEs	15 worker PEs	16 worker PEs	NumberOfPEs
Number of MAC units in each PE	64	64	MACunitsperPE
Total number of MAC units	960	1024	TotalMACunits
PE utilization	100.00% FC active compute-model utilization	86.82% FC systolic utilization	Both values use FC compute-model cycles, not full-program wall-clock or DMA/NoC idle time. The baseline reaches 100% mainly because its abstraction level is higher: FC operands are supplied from unconstrained controller-side and PE-local behavioral buffers, and DMA, NoC, data-staging stalls, and buffer-capacity limits are not included in this utilization denominator. Baseline system-level utilization is printed separately.
Local SRAM in each PE and capacity	No explicit PE-local SRAM; PE storage is modeled by behavioral buffers	64 KB per PE	LocalSRAMperPEbytes
Total on-chip SRAM capacity	0 explicit SRAM modules; controller/PE temporary storage uses higher-level behavioral buffers	8 MB global SRAM + 1 MB PE-local SRAM	Totalon-chipSRAMbytes
SRAM bit width	N/A for baseline SRAM because no explicit SRAM bus is modeled	32-bit word	SRAMbitwidth
Number of SRAM banks	0 explicit SRAM banks; banking is abstracted away by behavioral buffers	2 global SRAM banks; 64 input and 64 weight banks per PE	GlobalSRAMBanks, PElocalSRAMinput / weightbanks
DRAM read volume and DRAM write volume (GB)	0.25 GB read / 0.00 GB write (245,009,472 B / 4,000 B)	0.25 GB read / 0.00 GB write (245,009,472 B / 4,000 B)	Both designs read the same CNN tensors from DRAM and write 1000 final output scores.
DRAM bit width	32-bit AXI-like data word	32-bit AXI-like data word	RDATA/WDATA are <code>sc_uint<32></code> .
Number of DRAM banks	1 functional DRAM model	1 functional DRAM model	Current DRAM is not bank-accurate.
SRAM read volume and SRAM write volume (GB)	0.00 GB read / 0.00 GB write	0.31 GB read / 0.25 GB write (307,504,972 B / 250,410,124 B)	Baseline has no explicit SRAM hierarchy and uses behavioral temporary buffers. Optimized SRAM traffic is measured from the global scratchpad counters.
Total memory access volume (GB)	0.25 GB	0.80 GB	DRAM + SRAM traffic. Optimized includes explicit SRAM traffic that the baseline behavioral buffers do not model.
Execution cycles	138,725,673 simulated cycles (1,387,256,730 ns at 10 ns/cycle)	152,080,072 simulated cycles (1,520,800,720 ns at 10 ns/cycle); SRAM wait 139,478,774 cycles; FC systolic wait 65,940 cycles	Global SystemC timestamp plus optimized modeled SRAM and systolic cycle counters.
SystemC execution time	50 min	90 min	Wall-clock runtime measured externally; this is different from the SystemC simulated timestamp.
NoC routing method	Escape-adaptive minimal mesh routing with deterministic XY escape path	Escape-adaptive minimal mesh routing with deterministic XY escape path and HOST controller port	Two virtual channels are modeled: VC0 is the deadlock-free XY escape channel, and VC1 uses adaptive minimal routing.
Multicast support	Broadcast packet for shared input payloads	Row-wise systolic input broadcast	Multicastsupport
NoC protocol	Custom valid/ack packet NoC	Custom valid/ack packet NoC	NoCprotocol
NoC utilization	99.98% payload packing (61,900,669 / 61,911,008 words); 8.00 payload words/cycle	97.84% payload packing (76,789,835 / 78,483,152 words); 7.83 payload words/cycle	Payload packing uses 16 words/flit. Payload words/cycle uses controller injection cycles.
NoC transfer latency	2.00 cycles/flit	2.00 cycles/flit	Measured at controller packet injection interface.
Dataflow design	Behavioral controller-staged PE dispatch	SRAM-resident FC with two 2×4 systolic MVM sub-arrays	Baseline uses controller-side behavioral staging; optimized uses explicit SRAM storage and PE-level systolic FC mapping.

Continued on next page

TABLE III: Comparison between baseline and optimized designs (continued).

Metric / Design Aspect	Baseline Design	Optimized Design	Notes / Log Field
Time breakdown of different operations	DMA + NoC + PE compute, mostly sequential	DMA, SRAM, NoC, systolic compute, writeback with partial overlap	Use printed DMA, SRAM, NoC, and systolic counters.
Overlap of computation, communication, and memory access	Mostly serialized controller staging: DMA read, PE preload through NoC, compute dispatch, and result collection are issued in order	Limited FC overlap: next FC weight tile is prefetched from DRAM to SRAM while the current systolic tile computes	Baseline trace messages such as <code>PreloadingFClayer...workerPE...</code> indicate sequential PE preload before compute. Optimized overlap is explicit ping-pong weight prefetch, not full-program overlap.
Other design-specific metrics	714,188,480 baseline PE MAC ops; 120 PE compute jobs; 58,621,952 baseline FC MAC ops; 45 FC PE compute jobs; No explicit SRAM hierarchy; controller-side behavioral feature buffer with effectively unconstrained temporary storage	58,621,952 FC systolic MAC ops; 2,173 FC input tiles; Global SRAM peak used 1,500,384 words; DMA-to-SRAM descriptors; bounded 8-word controller tile registers; PE-local SRAM and nonlinear module	Highlights hardware meaning of optimized storage.